PySpark Skill Booster Interview Prep

Sachin Chandrashekhar

Founder, Data Engineering Hub™

https://sachinchandrashekhar.com

Follow us on social media for exciting updates and exclusive content!

YouTube: https://www.youtube.com/@MentorSachin

Instagram: https://www.instagram.com/mentorsachin/

Website: https://dataengineeringhub.in

legal@dataengineeringhub.in www.dataengineeringhub.in

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

Copyright & Usage Notice: This document and its supporting artifacts are copyrighted and intended strictly for personal use only.

- This document is reserved exclusively for individuals enrolled in Data Engineering courses by Sachin Chandrashekhar.

- Please do not share this document or upload it publicly. It is intended for personal use only.

- If you have obtained this document for free from any website other than the official course website, or from social media or public git repositories, please contact **legal@dataengineeringhub.in**

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

⚡ Quick Syntax Reference

📊 DataFrame Creation & Schema

Operation	Syntax
Read CSV with schema inference	`spark.read.option("header", "true").option("inferSchema", "true").csv(path)`
Define explicit schema (DDL)	`schema = "col1 INT, col2 STRING, col3 DOUBLE"`
Read with explicit schema	`spark.read.option("header", "true").schema(schema).csv(path)`
Show DataFrame	`df.show(5)`
Print schema	`df.printSchema()`
Count rows	`df.count()`
Get column names	`df.columns`

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

💾 Reading & Writing

Operation	Syntax
Write Parquet (overwrite)	`df.write.mode("overwrite").parquet(path)`
Write Parquet (append)	`df.write.mode("append").parquet(path)`
Read Parquet	`spark.read.parquet(path)`
Read with NULL handling	`spark.read.option("nullValue", "NULL").csv(path)`

🔄 Transformations

Operation	Syntax
Select columns	`df.select("col1", "col2")`
Select with col()	`df.select(col("col1"), col("col2"))`
Filter single condition	`df.filter(col("country") == "USA")`
Filter with AND	`df.filter((col("x") > 100) & (col("y") >= 2))`
Filter with OR	`df.filter((col("x") == "A") | (col("x") == "B"))`
Add column (constant)	`df.withColumn("status", lit("Active"))`
Add column (calculated)	`df.withColumn("tax", col("price") * 1.1)`
Conditional column	`df.withColumn("cat", when(col("x") > 100, "High").otherwise("Low"))`
Drop column	`df.drop("col_name")`
Rename column	`df.withColumnRenamed("old", "new")`
Chain transformations	`df.filter(...).withColumn(...).select(...)`

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

📊 Aggregations

Operation	Syntax
Group by with count	`df.groupBy("col").count()`
Sum	`df.groupBy("col").agg(sum("amount").alias("total"))`
Average (rounded)	`df.groupBy("col").agg(round(avg("amt"), 2).alias("avg"))`
Min and Max	`df.groupBy("col").agg(min("price"), max("price"))`
Multiple aggregations	`df.groupBy("col").agg(count("id"), sum("amt"), avg("val"))`
Count distinct	`df.agg(countDistinct("col").alias("unique"))`
Sort ascending	`df.orderBy("col")`
Sort descending	`df.orderBy(col("col").desc())`
Aggregate without groupBy	`df.agg(count("id"), sum("amt"), avg("val"))`

🔗 Joins

Operation	Syntax
Inner join	`df1.join(df2, df1.id == df2.id, "inner")`
Left join	`df1.join(df2, df1.id == df2.id, "left")`
Right join	`df1.join(df2, df1.id == df2.id, "right")`
Chain multiple joins	`df1.join(df2, ..., "inner").join(df3, ..., "inner")`
Find unmatched records	`df1.join(df2, ..., "left").filter(col("df2_col").isNull())`

❓ NULL Handling

Operation	Syntax
Filter NULL values	`df.filter(col("col").isNull())`
Filter NOT NULL	`df.filter(col("col").isNotNull())`
Drop rows with NULL	`df.dropna(subset=["col"])`
Replace NULL with value	`df.fillna({"col": 0})`

🔍 Other Operations

Operation	Syntax
Distinct values	`df.select("col").distinct()`
Drop duplicates (all cols)	`df.dropDuplicates()`
Drop duplicates (specific)	`df.dropDuplicates(["col"])`
Uppercase	`df.withColumn("upper", upper(col("name")))`
Lowercase	`df.withColumn("lower", lower(col("name")))`
Concatenate	`df.withColumn("full", concat(col("a"), lit("-"), col("b")))`
Split string	`df.withColumn("parts", split(col("email"), "@"))`
Extract from split	`df.withColumn("domain",`

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

	split(col("email"), "@")[1])

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

💼 Interview Questions & Answers

🎯 Core Concepts

Q1: What is PySpark and why use it?

✅ PySpark is the Python API for Apache Spark, a distributed computing framework. Use it to process large datasets that don't fit in memory, leveraging multiple machines for parallel processing. It's 100x faster than MapReduce and is the standard for AWS Glue ETL jobs.

Q2: Why use explicit schemas in production instead of schema inference?

✅ Schema inference requires Spark to scan the entire file to determine data types, which is inefficient for large files (GBs/TBs). Explicit schemas improve performance by skipping this step and ensuring data type consistency.

Q3: Why is Parquet the preferred format for data lakes?

✅ Four key reasons:

- 1. Columnar storage - faster queries when selecting specific columns
- 2. Compression - smaller file sizes, lower storage costs
- 3. Schema embedded - no need to specify schema when reading
- 4. Optimized for analytics - better performance for aggregations/filters

Q4: What is a DPU in AWS Glue?

✅ DPU (Data Processing Unit) is Glue's unit of processing power. 1 DPU = 4 vCPU + 16 GB memory. You pay per DPU-hour in serverless pricing model.

🔄 Transformations

Q5: What are the different write modes in PySpark?

✅ Four modes:

- • overwrite - Replace existing data completely
- • append - Add new data to existing data
- • ignore - Skip writing if data exists
- • error (default) - Throw error if data exists

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

Q6: How do you add a column with a constant value?

✅ Use withColumn() with lit() function:

```
df.withColumn("status", lit("Active"))
```

Q7: How do you create conditional columns?

✅ Use when/otherwise:

```
df.withColumn(
    "category",
    when(col("amount") > 200, "High")
    .when(col("amount") > 100, "Medium")
    .otherwise("Low")
)
```

Q8: How do you chain multiple transformations?

✅ Use backslash for line continuation:

```
result = df \
    .filter(col("amount") > 100) \
    .withColumn("discount", col("amount") * 0.1) \
    .select("id", "amount", "discount")
```

🔗 Joins

Q9: Explain different join types

✅ Join types:

- • inner - Only matching rows from both DataFrames
- • left - All rows from left + matching from right (NULLs for non-matches)
- • right - All rows from right + matching from left (NULLs for non-matches)
- • full/outer - All rows from both (NULLs where no match)

Q10: How do you find customers without orders?

✅ Use left join + filter for NULL:

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

```
customers_without_orders = customers_df.join(
    orders_df,
    customers_df.customer_id == orders_df.customer_id,
    "left"
).filter(col("order_id").isNull())
```

❓ NULL Handling

Q11: What's the difference between NULL literal "NULL" and actual NULL?

✅ In CSV files, "NULL" is a string, not an actual NULL value. Use option("nullValue", "NULL") when reading to convert "NULL" strings to actual NULLs.

Q12: How do you handle NULL values in PySpark?

✅ Three approaches:

- 1. Identify: isNull() or isNotNull()
- 2. Exclude: dropna(subset=["col"])
- 3. Replace: fillna({"col": 0})

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

📊 Aggregations

Q13: How do you perform multiple aggregations in one operation?

✅ Use agg() with multiple functions:

```
df.groupBy("category").agg(
    count("id").alias("total_count"),
    sum("amount").alias("total_revenue"),
    avg("amount").alias("avg_value"),
    max("amount").alias("max_value")
)
```

Q14: How do you round decimal values in aggregations?

✅ Use round() function:

```
df.groupBy("customer_id").agg(
    round(avg("amount"), 2).alias("avg_order")
)
```

🔍 Other Operations

Q15: What's the difference between distinct() and dropDuplicates()?

✅ Both remove duplicates:

- • distinct() - Works on selected columns only
- • dropDuplicates() - Can specify which columns to check or check all columns
- • dropDuplicates(["col"]) - Keeps first occurrence based on specific column(s)

Q16: How do you extract domain from email address?

✅ Use split() and list indexing:

```
df.withColumn(
    "domain",
    split(col("email"), "@")[1]
)
```

Q17: How do you sort in descending order?

✅ Use desc() method:

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

```
df.orderBy(col("amount").desc())
```


© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

✨ Common Patterns & Best Practices

🎯 Production Best Practices

✅ Always use explicit schemas in production

✅ Use Parquet format for data lake storage

✅ Organize data into raw/ and curated/ folders

✅ Handle NULL values explicitly

✅ Use descriptive column aliases in aggregations

✅ Chain transformations for readability

✅ Stop Glue notebooks when not in use

🛠️ Troubleshooting Tips

🔧 Problem: fillna() not replacing NULL values

Solution: Add option("nullValue", "NULL") when reading CSV

🔧 Problem: Filter with AND/OR not working

Solution: Put each condition in separate parentheses: (cond1) & (cond2)

🔧 Problem: Column not found after join

Solution: Qualify column names: df1.col_name vs df2.col_name

🔧 Problem: Code has extra backslashes when pasted

Solution: Remove extra backslashes manually after paste

💰 Cost Optimization

⚠️ Set idle timeout to 30 minutes for notebooks

⚠️ Use minimum 2 workers for learning/testing

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

⚠️ Stop notebooks immediately after use

⚠️ Clean up S3 buckets and IAM roles after practice

⚠️ Monitor Glue job runs in console

📚 Common Imports

```
import boto3
from pyspark.sql.functions import (
    col, lit, upper, lower, concat, split,
    when, count, sum, avg, min, max, round,
    countDistinct
)
```


© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

✅ Quick Troubleshooting Checklist

Before asking for help, verify:

🔍 Schema & Data Issues:

- ☑️ Did you define explicit schema?
- ☑️ Are column names spelled correctly?
- ☑️ Did you handle NULL values properly?
- ☑️ Is data in expected S3 bucket/path?

🔍 Code Syntax:

- ☑️ Did you import required functions?
- ☑️ Are parentheses balanced?
- ☑️ Did you use backslash for line continuation?
- ☑️ Are quotes matched (single vs double)?

🔍 Glue Notebook:

- ☑️ Is session created (ran boilerplate code)?
- ☑️ Did you wait for cell execution to complete?
- ☑️ Are you using correct IAM role?
- ☑️ Did you stop notebook after use?

🎓 Final Interview Tips

📌 For 0-3 Year Experience:

- • Focus on coding questions - syntax and basic operations
- • Practice all exercises from the course
- • Be ready to write code on whiteboard or live coding
- • Know common functions and transformations

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

📌 For 3-7 Year Experience:

- • Expect scenario-based questions
- • Explain your approach and reasoning
- • Discuss performance optimization – Covered in Diamond Membership
- • Share real-world project experiences – Covered more Pyspark Based Production Grade Real Projects in Diamond membership

📌 For 7+ Years / Architect Level - Covered in Diamond Membership:

- • Focus on architecture and design decisions
- • Discuss trade-offs and best practices
- • Explain Spark internals (partitioning, shuffling, etc.)
- • Advanced optimization techniques

Follow us on social media for exciting updates and exclusive content!

YouTube: https://www.youtube.com/@MentorSachin

Instagram: https://www.instagram.com/mentorsachin/

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution